

Procedimiento de Restauración / Disaster Recovery — AMS

Autor: David Cabezas · **Fecha:** 2026-04-28 · **Última prueba:** _pendiente_

Este doc cubre cómo restaurar la BD de AMS desde un backup. **Un backup que no se probó restaurar no existe** — agendá una prueba trimestral.

1. Estado actual del sistema de backup

Capa	Estado	Ubicación	Frecuencia	Retención
Backup local	■ Activo	`/var/backups/ams/{daily,weekly,monthly}`	Diario 03:00 UTC	7d + 4w + 12m
Snapshot Hostinger	■ Verificar panel hPanel	proveedor	Semanal automático	según plan
Off-site Backblaze B2	■ Pendiente setup	(cuenta a crear)	Diario tras local	infinito
Backup binarios (fotos/audios)	■ No incluido aún	—	—	—

RTO actual (tiempo máximo para volver online): ~30 min si tenés acceso al VPS.

RPO actual (data loss máxima): ~24h (último backup diario).

2. Restaurar BD desde backup local (escenario más común)

```
```bash
```

### 1. SSH al VPS

```
ssh root@187.77.223.137
```

### 2. Listar backups disponibles

```
ls -lah /var/backups/ams/daily/
```

```
ls -lah /var/backups/ams/weekly/
```

```
ls -lah /var/backups/ams/monthly/
```

### 3. Detener el backend (deja el frontend para mantener UX "en mantención" si querés)

```
cd ~/ASSET-MANAGEMENT-SOFTWARE
```

```
docker compose stop ocp-backend
```

## 4. Backup de seguridad de la BD actual ANTES de restaurar

```
DB="/var/lib/docker/volumes/ocp_db_data/_data/ocp_maintenance.db"
cp "$DB" "${DB}.before_restore_$(date +%Y%m%d_%H%M%S)"
```

## 5. Descomprimir el backup elegido a un archivo temporal

```
BACKUP_FILE="/var/backups/ams/daily/ams_20260428_030000.db.gz" # ← cambiá esto
gunzip -c "$BACKUP_FILE" > /tmp/ams_restore.db
```

## 6. Verificar integridad ANTES de copiar sobre la BD productiva

```
sqlite3 /tmp/ams_restore.db "PRAGMA integrity_check;"
```

**debe imprimir: ok**

## 7. Reemplazar la BD

```
mv /tmp/ams_restore.db "$DB"
chown ubuntu:ubuntu "$DB"
chmod 664 "$DB"
```

## 8. Levantar backend y verificar

```
docker compose start ocp-backend
docker compose logs -f ocp-backend # esperar "Application startup complete"
```

## 9. Smoke test desde otra terminal

```
curl -fsS https://mageam.com/api/v1/health
```

**debe responder OK**

...

**Tiempo estimado: 5-15 min.**

---

### 3. Setup pendiente: Off-site con Backblaze B2

#### 3.1 Crear cuenta B2

1. Ir a <https://www.backblaze.com/b2/sign-up.html>
2. Free tier: **10 GB free** + 1 GB/día download. Suficiente para nuestros ~13 MB diarios × 30 = 400 MB/mes.
3. Crear bucket `ams-backups` (privado, sin lifecycle al inicio).
4. Crear **Application Key** scoped al bucket. Guardar `keyID` + `applicationKey` .

#### 3.2 Instalar y configurar rclone en el VPS

```
``bash
```

```
ssh root@187.77.223.137
```

### Instalar rclone

```
curl https://rclone.org/install.sh | sudo bash
```

### Configurar el remote "b2"

```
rclone config
```

```
> n (new remote)
```

```
> name: b2
```

```
> storage: backblaze (option 7 aprox)
```

```
> account: <keyID>
```

```
> key: <applicationKey>
```

```
> endpoint: (vacío, usa default)
```

```
> q (quit config)
```

### Verificar

```
rclone lsd b2:
```

**debe listar el bucket "ams-backups"**

## Probar copia manual

```
rclone copy /var/backups/ams/daily/ b2:ams-backups/daily/ --dry-run -v
```

**si OK, sin --dry-run para subir**

...

A partir del próximo cron job (03:00 UTC), `ams_backup.sh` detectará `b2:` configurado y subirá automáticamente.

### 3.3 Verificar off-site

```
```bash
```

Ver lo que hay en B2

```
rclone ls b2:ams-backups/daily/ | head -5
```

Tamaño total

```
rclone size b2:ams-backups/
```

...

Costo estimado: **<1 USD/mes** (free tier cubre primeros 10 GB).

4. Restaurar desde Backblaze B2

```
```bash
```

### 1. SSH al VPS (o nuevo VPS si el original murió)

```
ssh root@<IP>
```

### 2. Listar backups en B2

```
rclone ls b2:ams-backups/daily/ | sort -k2 | tail -20
```

### 3. Bajar el más reciente

```
rclone copy b2:ams-backups/daily/ams_20260428_030000.db.gz /tmp/
```

## 4. Continuar con el procedimiento §2 desde paso 5.

...

---

## 5. Restauración total (VPS muerto)

Escenario peor caso: VPS Hostinger inaccesible o destruido.

### 5.1 Pre-requisitos

- Acceso a Hostinger panel (rebuild VPS) **O** cuenta lista en Hetzner / Vultr para VPS nuevo.
- Credenciales B2 a mano.
- Acceso al repo GitHub [ValueStrategyConsulting/AMS-Production](#) .

### 5.2 Pasos

```bash

1. Provisionar VPS nuevo (Ubuntu 22.04 LTS, 4 GB RAM mínimo)

2. Setup base

```
apt-get update && apt-get install -y docker.io docker-compose-v2 sqlite3 git curl
systemctl enable docker
```

3. Clonar repo

```
git clone https://github.com/ValueStrategyConsulting/AMS-Production.git
~/ASSET-MANAGEMENT-SOFTWARE
cd ~/ASSET-MANAGEMENT-SOFTWARE
git checkout feature/multi-plant # o la rama actual
```

4. Setup .env

```
cp .env.example .env
nano .env # configurar SECRET_KEY, DB paths, etc.
```

5. Levantar containers (sin BD primero)

```
docker compose up -d ocp-backend  
sleep 10  
docker compose stop ocp-backend
```

6. Bajar último backup de B2

```
curl https://rclone.org/install.sh | bash  
rclone config # configurar b2 con credenciales  
rclone copy b2:ams-backups/daily/$(rclone ls b2:ams-backups/daily/ | sort -k2 | tail -1 | awk '{print $2}')  
/tmp/
```

7. Restaurar (igual que §2 paso 5-9)

8. Apuntar DNS al nuevo IP

(en Cloudflare / DNS provider, cambiar A record de mageam.com)

9. Smoke test completo

...

RTO escenario total: ~2-4h (incluyendo provisión + DNS propagation).

6. Plan de prueba trimestral

Cada 3 meses, ejecutar prueba **no destructiva** en staging:

1. Levantar VPS staging temporal (Hostinger o Hetzner free tier).
2. Restaurar último backup B2 ahí.
3. Verificar:
 - [] App levanta sin errores
 - [] Login funciona
 - [] Datos OT/avisos están presentes (count cualquiera de yer)
 - [] Audit log preserva entries
4. Documentar tiempo total + cualquier problema.
5. Apagar VPS staging.

Última prueba ejecutada: _pendiente — agendar para próximo trimestre_.

7. Backups de binarios (fotos, audios) — pendiente

Hoy las fotos van al volumen `ocp_db_data/capture_photos` . NO están en el backup.

TODO: extender `ams_backup.sh` con:

```
```bash
```

## Agregar antes de la sección §6

```
PHOTOS_DIR="/var/lib/docker/volumes/ocp_db_data/_data/capture_photos"
```

```
PHOTOS_BACKUP="${DAILY_DIR}/photos_${DATE_STAMP}.tar.gz"
```

```
tar -czf "$PHOTOS_BACKUP" -C "$(dirname $PHOTOS_DIR)" "$(basename $PHOTOS_DIR)"
```

```
rclone copy "$PHOTOS_BACKUP" "${RCLONE_REMOTE}/photos/"
```

```
```
```

Estimado: ~5-10 GB/año en producción. Cabe en B2 free tier.

Esfuerzo: 30 min. Pendiente para la próxima sesión.

8. Monitoreo: alertar si falla

El backup actual logea a `/var/log/ams_backup.log` pero **no alerta**. Si falla 3 días seguidos, no nos enteramos.

Recomendado (~30 min setup):

1. Cuenta UptimeRobot free.
 2. Heartbeat URL (servicio gratis tipo healthchecks.io).
 3. Modificar `ams_backup.sh` para hacer `curl https://hc-ping.com/<UUID>` al final si todo OK.
 4. Si UptimeRobot/healthchecks no recibe el ping → email de alerta.
-

9. Comandos útiles para diagnóstico rápido

```
```bash
```

## ¿El cron corrió ayer?

```
grep "$(date -d yesterday +%Y-%m-%d)" /var/log/ams_backup.log
```

## ¿Cuánto pesa el último backup?

```
ls -lah /var/backups/ams/daily/ | tail -1
```

## ¿El B2 está sincronizado?

```
rclone size b2:ams-backups/
```

## ¿La BD está sana?

```
sqlite3 /var/lib/docker/volumes/ocp_db_data/_data/ocp_maintenance.db "PRAGMA integrity_check;"
```

## ¿Cuándo fue el último write a la BD?

```
stat -c '%y' /var/lib/docker/volumes/ocp_db_data/_data/ocp_maintenance.db
```

## ¿Cron está activo?

```
systemctl is-active cron && cat /etc/cron.d/ams_backup
```

```
...
```

---

**Versión:** 1.0 · Próxima revisión: tras primera restauración exitosa.